

CityTech TTP Summer Bootcamp

Intro to Databases and SQL

2026-06-22

Agenda

1. Review and Follow-Ups
2. What Is a Database?
3. Database Terminology
4. SQL and Querying
5. SQL Flavors and SQLite
6. Code Along: Exploring a Database
7. Relational Concepts: Keys and Joins
8. SQL Games

Review

What are the most important things you remember from last week?

Follow Up: Continue Building!

Proud of the project you finished last week? **Keep building it** and showcase it as a **personal project**.

- You can keep going **as a pair**: that shows you can **collaborate and communicate**
- Prefer to make it your own? **Fork the repo** and add an honest README disclaimer: it was a **collaborative** project before it was forked, and **solo** after

What Is a Database?

A **database** stores data **permanently**, so it survives even after your program or server stops.

- Memory (**RAM**) is **volatile**: it is wiped on restart. Disk is **non-volatile**: it persists
- The **Express servers** you built last week kept data in RAM, so it was lost on every server **restart**
- A database keeps your data on disk, and lets you **store, search, update, and relate** lots of it efficiently
- Most apps rely on one **to provide** features like accounts, posts, orders, and messages

Database Terminology

The words we use to describe data.

Term: Table

A **table** holds one kind of thing, organized into rows and columns, like a spreadsheet.

id	name	major
1	Ada	CS
2	Linus	CS
3	Grace	Math

The `students` table.

Term: Database

A **database** is an organized collection of related **tables**, stored together.

school database

students

courses

enrollments

Term: Column (or Field)

A **column** (also called a **field**) is one **attribute** that every row has, like `major`.

id	name	major
1	Ada	CS
2	Linus	CS
3	Grace	Math

Term: Row (or Record)

A **row** (also called a **record**) is one **entry**: all the fields for a single item.

id	name	major
1	Ada	CS
2	Linus	CS
3	Grace	Math

Term: Cell (or Value)

A **cell** holds a single **value**: one field for one row.

id	name	major
1	Ada	CS
2	Linus	CS
3	Grace	Math

The highlighted cell is Linus's `major` : `"CS"` .

Term: Schema

A **schema** is the **structure** of your data: the tables, their columns, and each column's type.

```
CREATE TABLE students (  
  id      INTEGER PRIMARY KEY,  
  name    TEXT,  
  major   TEXT  
);
```

It is the **blueprint**; the rows are the data that fills it in.

SQL

SQL (Structured Query Language) is how you talk to a relational database.

- You write **queries** to create, read, update, and delete data (**CRUD**)
- Reminder: **CRUD** = **C**reate, **R**ead, **U**ppdate, **D**eleate, the four basic data operations
- It's **declarative**: you describe *what* you want, and the database works out *how* to get it

Term: Query

A **query** is a request to the database, written in SQL.

```
SELECT name, major FROM students WHERE major = 'CS';
```

Returns:

name	major
Ada	CS
Linus	CS

Common SQL Commands

The four CRUD operations map to four SQL keywords:

```
SELECT name FROM students;  
INSERT INTO students (name) VALUES ('Ada');  
UPDATE students SET major = 'Math' WHERE id = 2;  
DELETE FROM students WHERE id = 3;
```

- **SELECT** = Read, **INSERT** = Create, **UPDATE** = Update, **DELETE** = Delete

The Structure of a Query

Like any SQL statement, a query is a **verb** plus a set of **clauses**.

```
SELECT name, major    -- verb: the action (here, read these columns)
FROM students         -- clause: which table
WHERE major = 'CS'    -- clause: which rows
ORDER BY name;        -- clause: how to sort
```

- It opens with a **verb**: `SELECT`, `INSERT`, `UPDATE`, or `DELETE`
- Then come **clauses**, each introduced by a **keyword** like `FROM`, `WHERE`, or `ORDER BY`
- Clauses follow a set order, and the statement ends with a semicolon `;`

Required and Optional Clauses

Some clauses are **required**, others are **optional**.

- **SELECT** and **FROM** are the core: say *what* to read and *where* from
- **WHERE**, **ORDER BY**, **LIMIT**, etc are **optional**: add them only when needed
- **Order matters** Clauses must appear in a fixed sequence (**SELECT** , then **FROM** , then **WHERE** , then **ORDER BY**), or it is a syntax error

More on SQL

SQL is a **standard language**, so the same core skills work across many databases.

- **Learn it once:** your skills transfer across most relational databases
- Expect small **dialect** differences from one database to another

Resources to Learn SQL

Official documentation:

- **PostgreSQL:** <https://www.postgresql.org/docs/>
- **SQLite:** <https://www.sqlite.org/docs.html>

Tutorials and practice:

- **W3Schools SQL:** <https://www.w3schools.com/sql/>
- **SQLBolt** (interactive lessons): <https://sqlbolt.com/>

Flavors of SQL

Different databases implement SQL with small **dialect** differences:

- **PostgreSQL**: powerful, open-source, standards-focused; a great default (installed in Week 1)
- **SQLite**: a tiny, file-based database; zero setup, ideal for small apps and learning
- **MySQL / MariaDB**: very common in web stacks
- **Big vendors** offer their own: **SQL Server** (Microsoft) and **Aurora / RDS** (AWS)

The **core SQL** is the same; details like data types and extra features vary.

Installing SQLite

SQLite is tiny and needs almost no setup.

macOS (often already installed; check with `sqlite3 --version`):

```
brew install sqlite
```

Windows (WSL / Ubuntu):

```
sudo apt update && sudo apt install sqlite3
```

Both give you the `sqlite3` command to open and explore a database file.

Code Along: Exploring a SQL Database

We'll explore a SQLite database of **video games** together.

Download **video_games.sqlite** from the class repo, then open it:

```
sqlite3 video_games.sqlite
```

- **.tables** lists the tables; **.schema games** shows a table's structure
- These **.**-commands are **SQLite shell** commands, not standard SQL
- **SELECT * FROM games;** reads every row; then filter with **WHERE** and sort with **ORDER BY**

Relational Databases

The databases we've used are **relational**: data lives in **tables** that **relate** to each other.

- Each table holds one kind of thing (`games` , `studios`)
- Tables link through shared values: a game's `studio_id` points to a row in `studios`
- This avoids repeating data and keeps it consistent
- You query them with **SQL**

Relational vs Non-Relational

Not every database is relational. **Non-relational** ("NoSQL") databases store data as documents, key-value pairs, or graphs.

	Relational (SQL)	Non-Relational (NoSQL)
Structure	tables with a fixed schema	flexible documents / key-value
Strengths	consistency, relationships, rich queries	scale, flexibility, fast simple reads
Examples	PostgreSQL, SQLite, MySQL	MongoDB, Redis, DynamoDB

We focus on **relational** databases. Non-relational is **out of scope** for this program.

Primary Key

A **primary key** uniquely identifies each row in a table, like `id`.

id	name
2	Valve
6	FromSoftware

The `studios` table: no two studios share an `id`.

Foreign Key

A **foreign key** is a column that **points to the primary key of another table**, linking them.

games

id	title	studio_id
8	Portal 2	2
6	Elden Ring	6

studios

id	name
2	Valve
6	FromSoftware

`games.studio_id` (foreign key) points to `studios.id` (primary key): Elden Ring's `6` is FromSoftware, Portal 2's `2` is Valve.

Join

A **join** combines rows from two tables using their key relationship.

```
SELECT games.title, studios.name  
FROM games JOIN studios  
ON games.studio_id = studios.id;
```

title	name
Elden Ring	FromSoftware
Portal 2	Valve

SQL Games

Practice SQL by playing. **Start with #1, and don't move on until we do it together as a class.**

1. **SQL Noir:** <https://www.sqlnoir.com/>
2. **SQL Police Department:** <https://sqlpd.com/>
3. **SQL Murder Mystery:** <https://mystery.knightlab.com/>